Decentralized Intelligence Track

INDEX / Problem Statements

Code	Statement	Teams	Page No.
DI-01	Debug Dojo		2
DI-02	NeuroScope 3D		8
DI-03	LiveSub		14
DI-04	Chimney Census		21
DI-06	Pay-per-call Agent Economy		25

Page numbers refer to the combined document, with this index on page 1 and each problem statement following in the order listed.

Debug Dojo

A coding coach that helps you find the bug without handing you the answer. AI-powered education and learning technology · live demo + judge test pack · 24 hours.

The story

Rohan is in his first semester. His function should return the total of the last k marks in a list. It works on all three of his examples. The practice site says: Wrong Answer on test 5.

Test 5 is hidden. Yesterday the site showed him IndexError: list index out of range, which told him where his program died but not why. So he does what everyone in his hostel does: pastes the question into a chatbot, copies the answer and submits. Accepted.

Next week the exam has the same kind of loop, and there is no chatbot in the exam hall.

Rohan never needed the answer. He needed someone to point him to the right place to look, then a little more if he was still stuck, and next week a new problem that makes him face the same mistake until he stops making it.

Problem overview

Beginners learn programming by writing code and reading what goes wrong, but feedback is often hard to use. A traceback names an error without explaining it, and a program can run and still give the wrong answer. Tools that report only pass or fail don't show where to look; tools that give full answers too early remove the struggle that makes learning stick. And running learner code is risky: code from an unknown author must not read sensitive files, reach the network or consume unlimited resources.

Build a practice environment that runs learner code safely, explains failures in plain language, gives hints that get more specific without giving the answer away, and learns which mistakes each learner keeps making.

The challenge

Build a practice environment for one programming language your team can support well. Every submission must do all of the following:

1. **Challenges.** At least 10 challenges across at least 3 difficulty levels and at least 4 concepts. Each has a description, starter code, concept tags, at least 3 visible tests and at least 5 hidden tests, including edge cases. Your set must include the four judge challenges below, exactly as specified.

2. **Restricted execution.** Learner code never runs unrestricted. Full sandboxing is not required; a constrained local test runner is acceptable, and stronger isolation scores higher.
3. **Per-test outcomes.** Report every test separately as exactly one of: passed, wrong output, runtime error or timeout. A run killed by a memory, output or disk limit counts as a runtime error and says which limit.
4. **Hidden tests stay hidden.** Never reveal a hidden test's input, expected output, the program's output on it or its exception message. An explanation may name the kind of input involved ("a case where k is larger than the list"), never the values.
5. **Plain-language explanations.** When a submission fails, explain the failure in plain language to help the learner find the problem.
6. **Progressive hints.** Offer a ladder of at least 3 hints per failure, each more specific than the last. No hint, including AI-generated ones, may reveal the complete solution, even when the learner asks for it.
7. **Recurring mistakes.** Tag each failed submission with mistake categories. Your taxonomy must include these five, kept distinct, and may add more: off_by_one, unhandled_short_input (empty or too-short input), print_not_return, boundary_comparison (such as < for ⩽) and mutates_input. A category is recurring once it appears in at least 2 different challenges. Show it to the learner, and choose or generate the next challenge to practise it, saying why.
8. **Skills by concept.** Keep a mastery estimate per concept, updated from the learner's work. A pass without hints counts for more than a pass after hints.
9. **Judge interface.** Expose POST /api/submit, taking {learner_id, challenge_id, code} and returning the outcomes and the explanation as JSON in the shape shown in the Example; POST /api/hint, taking {submission_id} and returning the next hint; and GET /api/learner/{learner_id}, returning the learner's recurring mistakes, mastery per concept, and the next challenge with the reason.

The four judge challenges (translate the signatures into your language):

Challenge	Signature	Returns	Inputs
last-k-total	last_k_total(marks, k)	The total of the last k marks, or of all marks if k is larger than the list	marks: a list of integers, possibly empty; k: an integer, 0 or more
count-passes	count_passes(marks)	How many marks are 40 or more	marks: a list of integers, possibly empty
first-n-squares	first_n_squares(n)	The list [1, 4, 9, …, n^2], or [] when n is 0	n: an integer, 0 or more

Challenge	Signature	Returns	Inputs
middle-three	middle_three(items)	The middle three items as a list, or the whole list if it has fewer than 3 items	items: a list of odd length, or with fewer than 3 items

Data and tools

Resource	What it gives you	Licence
Exercism Python track	Exercises with tests, concepts, prerequisites and difficulty	MIT
MBPP	About 1,000 entry-level Python problems with tests	CC BY 4.0
HumanEval	164 Python problems with unit tests	MIT
Judge0	Self-hosted code execution, 90+ languages	GPL-3.0
Piston	Self-hosted code execution engine (its public API now needs a key, so self-host it)	MIT
nsjail	Linux process isolation	Apache-2.0
Pyodide	Python in the browser via WebAssembly	MPL-2.0
Ollama	Run open-weight language models locally; any model API is fine too	MIT

Scoring

The judges run hostile programs, outcome-label checks, hidden-test secrecy checks and a scripted learner through your API before your slot. During the 15-minute slot they test hints and explanations and use your interface.

Test	What the judges do	Full marks when
Sandbox red-team	Submit about 15 hostile programs (network, files, secrets, infinite loops, memory, processes, output and disk floods, tricks against the test harness)	Every one is contained and never reported as passed; the server stays up; no secret or test file is exposed; the next normal submission runs correctly

Test	What the judges do	Full marks when
Outcome labels	Submit 4 programs per challenge for 3 challenges: pass, wrong output, error, run forever	All 12 labels are exactly right
Hidden-test secrecy	Submit programs that print or raise their inputs, request hints, inspect browser traffic	No hidden input, expected output, program output or exception message reaches the learner
Hint leakage	For 3 failing submissions, request every hint, including after asking for the answer; merge all code from the hints into the learner's code	The merged code still fails at least one hidden test
Recurring mistakes	A scripted learner submits about 11 programs over the 4 judge challenges	Both planted recurring mistakes are named, one-off slips are not, and the next challenge targets a recurring mistake with a reason
Explanations and hints	Two judges rate 5 failures, blind to team, 0–2 each on correct, specific and not giving the answer away	Points scale with the average rating

Example

Rohan submits this for last-k-total:

```
def last_k_total(marks, k):
    total = 0
    for i in range(len(marks) - k, len(marks)):
        total += marks[i]
    return total
```

Your /api/submit returns something like this:

```
{
  "submission_id": "s_0142",
  "learner_id": "rohan",
  "challenge_id": "last-k-total",
  "visible": [
    {"test": 1, "call": "last_k_total([10, 20, 30, 40], 2)", "expected":
"70", "got": "70", "outcome": "passed"},
```

```
    {"test": 2, "call": "last_k_total([5, 5, 5], 3)", "expected": "15",
"got": "15", "outcome": "passed"},
    {"test": 3, "call": "last_k_total([7], 1)", "expected": "7", "got": "7",
"outcome": "passed"}
  ],
  "hidden": [
    {"test": 1, "outcome": "passed"},
    {"test": 2, "outcome": "passed"},
    {"test": 3, "outcome": "runtime_error"},
    {"test": 4, "outcome": "wrong_output"},
    {"test": 5, "outcome": "wrong_output"},
    {"test": 6, "outcome": "passed"}
  ],
  "mistake_tags": ["unhandled_short_input"],
  "explanation": "All 3 visible tests pass, but 3 of the 6 hidden tests fail:
one crashes and two give the wrong total. All three are cases where k is
larger than the number of marks. Your loop starts at len(marks) - k, which is
negative then."
}
```

A good hint ladder for it:

1. Your three examples never ask for more marks than the list holds. What should last_k_total return when k is bigger than the list?
2. Work out last_k_total([2, 5], 3) by hand. Which index does your loop start at, and what does marks[-1] mean in Python?
3. In Python a negative index counts back from the end, so some marks get added twice. Your start index must never go below 0.
4. Python has a built-in that returns the larger of two numbers. Use it to work out the start index once, before the loop.

No hint contains code, and the learner still writes the fix.

Rules

- **Language and stack:** one programming language for learners; any stack for the platform.
- **Challenge content:** write your own, or adapt openly licensed sources with attribution. Write your own hidden tests.
- **AI:** any model or API may write explanations and hints. Log every prompt and response; judges may ask to see them.
- **Learner data:** pseudonymous accounts only. No real student data.

- **Judge access:** your API endpoints must be reachable by the judges from the start of judging.

Deliverables and judging

Hand in by the freeze:

1. A code repository with a README and one command that starts everything.
2. A live URL, or a laptop the judges can use, with the API endpoints.
3. Your mistake taxonomy and how each category is detected, on one page.
4. A 5-minute demo, then a viva.

Component	Points	How it is judged
Sandbox red-team	15	Share of hostile programs contained
Outcome labels and hidden-test secrecy	15	Exact labels; no leak of hidden tests
Hint leakage	10	Merged hint code never passes all hidden tests
Recurring mistakes and skills	10	Scripted learner result; mastery moves with the evidence
Explanations and hints	15	Blind judge ratings
Learning experience	15	Would a struggling beginner use it, and learn rather than copy?
Technical depth and viva	20	Explains the isolation, the test harness, the hint guardrails and the learner model without notes

NeuroScope 3D

See a brain tumour in 3D, measure it, and know how far to trust it. Health-tech · 3D medical image segmentation + web visualisation · live demo + judge test pack · 24 hours.

> Research and teaching prototype only. It must never present model output as a diagnosis, and it uses public, de-identified research data only.

The story

Ishita, a third-year BTech student, volunteers at a radiology teaching lab. Today's teaching case comes from a public research dataset: an adult glioma, scanned four different ways, each scan a stack of 155 slices.

The trainee beside her scrolls up and down for ten minutes, building the tumour in his head. Is it one mass or two? Which side is it on? How big is it, in millilitres?

A model can colour every voxel that looks like tumour. But a coloured blob helps only if it sits in the right place, on the right side, at the right size, and admits when it is unsure.

Build the tool Ishita wants: drop in a scan, watch the suspected tumour appear inside a 3D brain, measure it, and see exactly how much to trust it.

Problem overview

A brain MRI study is four co-registered 3D volumes: T1, T1Gd (T1 after a contrast agent), T2 and FLAIR. Expert labels mark three tissue types: necrotic core (NCR), oedema (ED) and enhancing tumour (ET). Models and metrics use three nested regions built from them:

Region	Made of
Whole tumour (WT)	NCR + ED + ET
Tumour core (TC)	NCR + ET
Enhancing tumour (ET)	ET

Build a web platform that takes one study, segments these regions with a 3D model, shows them in 3D and in linked slices, measures their volumes, and reports how accurate the model is and where it may fail.

The challenge

Every submission must do all of the following:

1. **Upload and validate.** Accept four NIfTI files (.nii or .nii.gz), one per sequence; the user or the filename says which file is which. Reject, with a clear message, any file that is unreadable, not 3D, not finite, constant (such as all zeros) or larger than 1 GB, and any set whose shapes or affines differ.
2. **Optional expert mask.** Accept an optional label file in BraTS 2021 codes (1 NCR, 2 ED, 4 ET). When it is present, show its regions and volumes next to the prediction.
3. **Segment live.** Run a pretrained or fine-tuned 3D model on the uploaded case. Produce WT, TC and ET masks mapped back to the original grid and affine. Also provide a command or endpoint that takes the four files and writes the predicted label map as NIfTI, in BraTS 2021 codes (1, 2, 4) on the input grid and affine; the judges' scorer reads it.
4. **3D view.** Show the brain surface and each region as meshes in millimetre (world) space. Users can rotate, zoom, toggle each region and change opacity.
5. **Linked slices.** Show axial, coronal and sagittal slices with the mask overlay and L/R, A/P and S/I labels. One crosshair links all three views and the 3D view, and its position is shown as voxel indices and world millimetres.
6. **Volume.** Report each region's volume in mL as voxel count × voxel volume from the header spacing, with the formula and the numbers shown.
7. **Evaluate.** Report Dice and sensitivity per region on at least 10 held-out labelled cases, split by patient. List the split, and disclose any overlap between your evaluation cases and your model's training data.
8. **Fail safely.** Handle broken, incomplete and unsupported inputs with clear messages, and warn or refuse when a set is clearly not a skull-stripped brain MRI. Never show a result for an input you could not process.
9. **Report.** Generate a report page or PDF with volumes, metrics, provenance (model name, version and licence; preprocessing; input SHA-256; code commit; timestamp), limitations and the disclaimer. The disclaimer also appears on every screen.

Extra credit: colour-coded agreement with the expert mask; uncertainty from test-time augmentation or an ensemble; finer out-of-distribution warnings; a quality warning instead of a rejection when a sequence is missing; HD95 in millimetres; modality-ablation explanations labelled as not clinical evidence; DICOM input; comparing two scans of one patient over time.

Out of scope: diagnosis, grading, genetic prediction, prognosis or treatment advice; surgical or radiotherapy planning claims; real patient scans; clinical-validation or regulatory claims.

Data and tools

Resource	What it gives you	Licence and access
BraTS 2021, Task 1 training set	1,251 adult glioma cases: T1, T1Gd, T2 and FLAIR plus expert labels (1 NCR, 2 ED, 4 ET). Co-registered, 1 mm^3, skull-stripped; 240 × 240 × 155	CC BY 4.0 on TCIA. The organizers hand out a 40-case dev subset
Medical Segmentation Decathlon, brain tumours	750 4D volumes (484 training with labels), from BraTS 2016–2017. Its channel order and label codes differ from BraTS 2021: read its dataset.json	CC BY-SA 4.0
MONAI brats_mri_segmentation	Pretrained SegResNet. Input: 4 channels at 1 mm, order T1Gd, T1, T2, FLAIR. Output: TC, WT, ET. Trained on BraTS 2018	Apache-2.0
Starter kit	The dev subset, a notebook that runs the bundle on one case, and the judges' scorer	Handed out at the start

Useful open-source libraries: MONAI, nibabel or SimpleITK, scikit-image, trimesh, NiiVue, Three.js, FastAPI.

Scoring

Runs that need only your command or endpoint (the accuracy, reproducibility, anisotropic and orientation cases) happen before your slot. The judges then check those results in your viewer and run the rest live in a 15-minute slot.

Test	What the judges do	Full marks when
Held-out accuracy	Run your pipeline on 10 hidden labelled cases and score Dice (WT, TC, ET) and sensitivity	Mean Dice within 0.05 of the organizers' baseline on the same cases, falling linearly to 0 at 0.20 below it. Sensitivity is scored the same way
Volume	Upload a case whose expert mask is a 20 × 20 × 20-voxel cube labelled 2, at spacing 1 × 1 × 2 mm	Expert-mask volumes read WT 16.0 mL (±1%), TC 0 and ET 0
Anisotropic case	Upload a hidden case, then the same case with every second axial slice removed	The two predicted WT volumes agree within 10%
Orientation, same anatomy	Upload a copy stored with its voxels flipped left–right and the affine changed to match	Tumour on the same labelled side in every view; WT volume within 2%
Orientation, mirrored anatomy	Upload a copy whose voxels are flipped left–right with the affine unchanged	The tumour moves to the other labelled side
Linked views	Click a bright marker block at a known position in one view	Crosshair lands on it in all views and in 3D, within 2 voxels
Failure pack	Upload 8 broken inputs: a non-NIfTI file named .nii.gz, a corrupt gzip, only three sequences, mismatched shapes, a 2D image, an all-zero volume, NaN values, a file over 1 GB	A clear message every time, no crash, no result
Reproducibility	Run the same case twice	Masks agree with Dice ⩾ 0.99
Blind case	Upload a case you have never seen, during the viva	The pipeline finishes with a plausible, case-specific mask
Input sanity	Upload a four-file set that is not skull-stripped, and one that is not a brain	A warning or a refusal

Example

Volume is voxel count times voxel volume, and 1 mL = 1,000 mm³:

Region	Voxels	Spacing (mm)	Volume
A predicted WT	15,000	1 × 1 × 1	15.0 mL
The same 15,000 voxels	15,000	1 × 1 × 3	45.0 mL
The judges' cube	8,000	1 × 1 × 2	16.0 mL

If the model predicts 12,000 WT voxels, the expert marked 10,000 and 9,000 overlap:

$$\text{Dice} = \frac{2 \times 9000}{12000 + 10000} = 0.818 \qquad \text{Sensitivity} = \frac{9000}{10000} = 0.90$$

Rules

- **Data:** public, de-identified research data only. No real patient scans. Never send scans to third-party services or language-model APIs.
- **Models:** any pretrained model public before the event, with its licence documented. Fine-tune only on the dev subset or the Decathlon set, and list the case IDs. At inference your pipeline may not read any stored dataset images or labels: a lookup disqualifies the team.
- **Hidden cases:** never train on, tune on or keep the cases the judges give you. Delete uploads automatically within 24 hours.
- **Live pipeline:** a pre-computed case is allowed only as a labelled backup.
- **Compute:** one case, from upload to report, within 2 minutes on the organizer GPU (T4 class).
- **Wording:** say "suspected tumour region predicted by the model" and "estimated volume". Never say "tumour detected", "diagnostic system" or "replaces the radiologist".

Deliverables and judging

Hand in by the freeze:

1. A code repository with a README and one command that starts the platform.
2. A live URL, or a laptop the judges can use.
3. One sample report generated by your platform.

4. A one-page model card: model, version, licence, training data, preprocessing and limits.
5. A 5-minute demo, then a viva.

Component	Points	How it is judged
Segmentation quality	25	Held-out accuracy: 20 for Dice, 5 for sensitivity
Correct 3D and linked views	15	Orientation and linked-view tests, smooth interaction
Volume	10	Volume and anisotropic tests
Explainability and traceability	10	Provenance, comparison with the expert mask, reproducibility
Reliability	10	Failure pack and input-sanity tests
User experience	10	Clear flow, usable by a non-expert
Responsible communication	10	No diagnostic claims, limits stated, correct terms
Viva	10	Explains preprocessing, channel order, volume maths and evaluation without notes

Data credit: cite BraTS 2021 (Baid et al., DOI 10.7937/jc8x-9874) and the Medical Segmentation Decathlon in your README if you use them.

LiveSub

Real-time multilingual subtitles for any YouTube video. Media and accessibility · browser extension + streaming speech AI · live demo + judge test pack · 24 hours.

The story

Aai is 68 and lives in Pune. Her neighbour keeps sending her Tamil cooking videos; she speaks Marathi and Hindi, and the videos have no captions at all.

Her grandson in Bengaluru watches Hindi stand-up full of English punchlines. YouTube's auto-captions turn half of every joke into nonsense, and auto-translate turns the rest into literal word salad. His flatmate is learning Tamil and wants the Tamil line and its English meaning on screen together.

All three are looking at the same YouTube player. None of them may download the video, and none of them should have to.

Put the right words on that player, in the right language, in time.

Problem overview

YouTube has auto-captions and auto-translate, but they fail in common situations:

- Captions are disabled or missing on many movies, uploads and regional-language videos.
- Auto-caption accuracy is poor for Indian languages, code-mixed speech such as Hinglish and Tanglish, accents, songs and noisy audio.
- Auto-translate is literal and loses idioms, tone and context.
- Viewers can't see two languages at once, for example the original plus a translation for language learning.

Build a Chrome extension that overlays accurate, synchronised subtitles on YouTube videos in the language the viewer wants, whether or not the video has captions:

1. **Detect** whether the video has a caption track.
2. **If it does,** use it as the source-language subtitles and offer translation into the viewer's language.
3. **If it doesn't,** capture the tab's audio, run speech-to-text and generate captions in the original language.
4. **Translate** (optional) with a neural machine translation model or a language model.

5. **Render** an overlay synced to playback, handling seek, pause and speed changes correctly.

The challenge

Every submission must do all of the following:

1. **Extension.** A Chrome extension (Manifest V3) that overlays subtitles on the YouTube player in normal, theatre and full-screen modes.
2. **Caption path.** When the video has a caption track, use it and translate it.
3. **Audio path.** When it doesn't, capture the tab audio and run speech-to-text. The viewer keeps hearing the video.
4. **Language detection.** Detect the spoken language automatically and show it; for code-mixed speech, show both languages.
5. **Two kinds of subtitles.** Source-language subtitles, and subtitles translated into the viewer's chosen language.
6. **Latency.** Near real time: each subtitle appears within 3–5 seconds of the end of its speech. Target: median at most 3 s and 90th percentile at most 5 s at 1× speed.
7. **Sync.** Subtitles stay correct through pause, seeking forwards and backwards, and speeds from 0.5× to 2×. Already-transcribed parts show their subtitles at once after a seek back.
8. **Style.** The viewer can change subtitle size, position and colour.
9. **Navigation.** YouTube changes video without reloading the page. Reset cleanly for each new video, and never attach speech from an ad to the video's timeline.
10. **Cue log for judges.** Export the session as JSON: one entry per subtitle with its start and end in video seconds, language, text, translation, source (captions or stt), and first_shown_at: the video time at which its final text first appeared in full. Also log every text change as revisions: a list of {at, text}.

Stretch goals, which earn extra credit:

- dual subtitles: original and translation shown together;
- Indian-language focus: Hindi, Marathi, Tamil, Telugu and code-mixed speech;
- on-device or offline mode for privacy, such as Whisper via WebGPU;
- context-aware translation that uses surrounding dialogue and the speaker's tone;
- speaker labels and non-speech cues such as [music] and [laughs];
- learning mode: click a word for its meaning and save it to a vocabulary list;
- export to .srt or .vtt;
- accessibility features for deaf and hard-of-hearing viewers.

Key technical challenges: capturing tab audio in Manifest V3, chunking streaming speech for low latency without cutting words mid-sentence, keeping subtitles in sync after seeks and speed changes, balancing accuracy against speed and cloud against on-device, and handling long videos without exhausting memory or API budgets.

Data and tools

Any speech-to-text or translation stack is allowed, including cloud APIs such as Google Speech-to-Text or Deepgram and language-model APIs. Open-source options:

Resource	What it gives you	Licence
Chrome tabCapture and the audio capture guide	Capturing tab audio in Manifest V3	Docs
Whisper, including large-v3-turbo	Multilingual speech-to-text with language detection	MIT
faster-whisper	Faster Whisper inference, with voice-activity filtering and word timestamps	MIT
whisper.cpp	Whisper in C/C++, with a WebAssembly build	MIT
Transformers.js	Whisper and other models in the browser, on WebGPU	Apache-2.0
IndicConformer 600M	Speech-to-text for Indian languages	MIT; accept the terms on Hugging Face first
Silero VAD	Voice-activity detection for cutting audio at pauses	MIT
IndicTrans2	Translation between English and the 22 scheduled Indian languages	MIT; Hugging Face weights need terms accepted
NLLB-200	Translation across 200 languages	CC BY-NC 4.0: non-commercial only
FLEURS, IndicVoices, Kathbath	Labelled speech in Indian languages, for testing your pipeline	CC BY 4.0
jiwer and sacreBLEU	WER, CER and chrF, the metrics the judges use	Apache-2.0

Scoring

The organizers play their own unlisted test videos, for which they hold exact transcripts, timings and English reference translations. The judges load your extension unpacked in a clean Chrome profile and export your cue log after each test.

Test	What the judges do	Full marks when
Accuracy	Play 2 minutes of each no-caption video (Hindi, Tamil, Marathi or Telugu, and English) at 1×	Error at or below the organizers' baseline in each language, falling linearly to 0 at 1.5 times the baseline's error. CER for Indian languages, WER for English, punctuation removed
Code-mixed speech	Play a Hinglish video	Two judges rate 10 lines from 0 to 2 for correct words in a readable script; points scale with the mean rating
Translation	Compare 10 subtitles translated into English with reference translations	chrF at or above the baseline (no credit at 0.7 times the baseline or below), and a mean meaning rating of 2 (0–2 per line)
Latency	Compute from the cue logs of the 1× runs	Median at most 3 s and 90th percentile at most 5 s. No credit at a median of 6 s or a 90th percentile of 10 s; linear in between, on the worse of the two
Sync	On the Hindi video: play 0:00–2:00 at 1×; pause 30 s; seek to 3:00 and play to 3:20; seek back to 0:30; seek to 3:40 and play 60 s at 1.5×, then 30 s at 0.75×	Median gap between each subtitle's start and the reference start at most 0.5 s (no credit at 2 s or more); after the seek back to 0:30, subtitles appear within 1 s; nothing from before a seek shows after it
Caption path	Play a video that has uploaded captions	Subtitles come from the caption track (source: captions), match its text, and the translation appears
Language detection	Play 4 short clips, one of them code-mixed	The right language for each; both languages for the code-mixed clip
Robustness	Open another video without reloading; listen during capture	Old subtitles are gone within 2 s, the tab stays audible, and an ad's speech never appears on the video's timeline

Example

Two sentences from an example Hindi video:

Sentence	Speech (video seconds)	First shown at (video seconds)	Lag
आज हम पनीर टिक्का बनाएंगे	12.40–14.10	16.30	2.20 s: within target
The next sentence	15.00–17.80	22.95	5.15 s: over 5 s

The first cue in your log:

```
{
  "start": 12.40,
  "end": 14.10,
  "lang": "hi",
  "text": "आज हम पनीर टिका बनाएंगे",
  "translation": "Today we will make paneer tikka",
  "source": "stt",
  "first_shown_at": 16.30,
  "revisions": [{"at": 16.30, "text": "आज हम पनीर टिका बनाएंगे"}]
}
```

Against the reference आज हम पनीर टिक्का बनाएंगे (5 words), the transcript has one wrong word, टिका for टिक्का: WER 1/5 = 20%, CER 2/25 = 8%.

At 1.5× speed, audio seconds are not video seconds. If an audio chunk started at video time 60.0 s and a word comes 4.0 s into that chunk:

```
t_{\text{video}} = 60.0 + 4.0 \times 1.5 = 66.0\ \text{s}
```

Rules

- **Platform:** a Chrome extension (Manifest V3), loaded unpacked for judging. The viewer never downloads files by hand; the extension may fetch model weights itself.
- **YouTube's terms:** don't download or redistribute video or audio. Process tab audio in memory as it plays, don't fetch the media stream separately, and export only text.
- **Privacy:** if audio leaves the browser for a cloud API, say so in the extension and let the viewer switch it off.
- **Keys:** no API keys in the repository or the extension package.

- **No lookups:** don't hard-code anything about the test videos, and don't look up or retrieve any dataset's transcripts or translations at run time. Training or fine-tuning on public data before the event is fine.

Deliverables and judging

Hand in by the freeze:

1. A code repository with a README, plus the extension as a .zip the judges can load unpacked.
2. Any backend running at a public URL for the whole judging window, or a laptop the judges can use.
3. A one-page architecture note: capture path, chunking, models, where audio goes, and how sync is kept.
4. A 5-minute demo, then a viva.

Component	Points	How it is judged
Transcription accuracy	15	10 for the mean accuracy score over the four languages, 5 for code-mixed speech
Translation	10	5 for chrF, 5 for the meaning ratings
Latency	10	Latency test
Sync	15	10 for the start-time gap, 5 for behaviour after seeks
Caption path, language detection and robustness	10	Caption path 4, language detection 3, robustness 3
Subtitle experience and stretch goals	20	Style controls, readability and the stretch goals delivered
Technical depth and viva	20	Explains capture, chunking, timing and the cost of long videos without notes

Chimney Census

Find and classify every brick kiln from space. Computer vision · oriented object detection on satellite imagery · leaderboard + judged brief · 24 hours.

The story

Every November, Inspector Farida Khan of a State Pollution Control Board opens the same file: 312 registered brick kilns in her district. Everyone in the office knows the real number is higher.

Since 2022, the rules have said old, smoky kilns must convert to cleaner designs such as zig-zag firing, and the deadlines have been pushed back twice. The kiln owners' association says most have converted. Farida has one jeep, two field staff and a district of 3,000 square kilometres.

Every few days, a European satellite photographs her whole district, free, at 10 metres per pixel. From that height a kiln is a smudge a few pixels long: an oval ring for an old Bull's Trench kiln, a circle for the rarer circular kind, a neat rectangle for a zig-zag.

A few pixels. That is all you get. Build the eyes Farida needs.

Problem overview

Detect every brick kiln in Sentinel-2 RGB tiles (128 × 128 pixels, 10 m per pixel), draw an oriented box around it, and classify it as one of three types:

Class	Kiln type	Seen from orbit
FCBK	Fixed Chimney Bull's Trench Kiln	Oval ring; the older, more polluting design
CFCBK	Circular FCBK	Circular ring; rare, about 3% of training kilns
Zigzag	Zig-zag kiln	Rectangle; the cleaner design the 2022 rules push kilns toward

You train on SentinelKilnDB: 2023–24 imagery of the Indo-Gangetic Plain, Pakistan, Bangladesh and Afghanistan. We test you on fresh 2025–26 imagery, partly from districts outside that coverage. Since your training images were taken, kilns have been built, demolished and converted.

The challenge

For every tile in the test set, output each kiln you find: its class, the four corners of its oriented box, and a confidence score.

- Corners are normalized to 0–1 (x to the right, y down) and listed in order around the box.
- A tile may hold zero, one or several kilns; most hold none.
- A kiln cut by a tile edge counts in that tile only if at least half of it lies inside.

Submit one CSV with one row per detection. Tiles with no detections have no rows.

```
tile_id,class,x1,y1,x2,y2,x3,y3,x4,y4,score
t_08812,Zigzag,0.405,0.305,0.495,0.282,0.507,0.329,0.417,0.352,0.91
```

Data

Pack	What is in it	Notes
Training: SentinelKilnDB	114,300 tiles holding 62,671 unique kilns (FCBK 33,963 · Zigzag 26,764 · CFCBK 1,944) and 41,068 kiln-free tiles	All three of its splits are yours to train on. Tiles overlap by 30 px, so one kiln can appear in two tiles (97,648 boxes in all). Labels come in YOLO-OBB, YOLO axis-aligned and DOTA formats; ~4 GB, also on Kaggle
Test tiles	~8,000 fresh 2025–26 tiles, same size and resolution	Random IDs, no coordinates
Demo District	~2,000 tiles of one district, with each tile's corner coordinates, plus that district's 2023–24 SentinelKilnDB labels	Not scored by mAP; for your Inspector's Brief
Starter kit	Data loader, a YOLO-OBB baseline notebook, the exact local scorer	Runs on a free T4 GPU

Scoring

The leaderboard metric is mean Average Precision at IoU 0.5, computed on rotated boxes and averaged over the three classes.

```
\text{mAP@0.5} = \tfrac{1}{3}\left(\text{AP}_{\text{FCBK}} +
\text{AP}_{\text{CFCBK}} + \text{AP}_{\text{Zigzag}}\right)
```

- Per class, all detections across all tiles are ranked by score. A detection is a true positive if its rotated IoU with an unmatched true kiln of the same class is at least 0.5.
- AP is the area under that class's precision–recall curve, with all-point interpolation as in the DOTA benchmark.
- Each class counts equally, so the rare circular kilns are a third of your score.
- Public leaderboard: 30% of test tiles. Final ranking: the other 70%, revealed at the closing.

Example

Tile t_08812 holds two kilns (hidden ground truth):

```
class,x1,y1,x2,y2,x3,y3,x4,y4
Zigzag,0.400,0.300,0.490,0.277,0.502,0.324,0.412,0.347
FCBK,0.722,0.699,0.808,0.749,0.778,0.801,0.692,0.751
```

Your rows for this tile:

```
tile_id,class,x1,y1,x2,y2,x3,y3,x4,y4,score
t_08812,Zigzag,0.405,0.305,0.495,0.282,0.507,0.329,0.417,0.352,0.91
t_08812,FCBK,0.727,0.697,0.813,0.747,0.783,0.799,0.697,0.749,0.66
t_08812,Zigzag,0.100,0.120,0.150,0.110,0.155,0.140,0.105,0.150,0.08
```

Notes:

- Row 1 overlaps the true zig-zag with rotated IoU ≈ 0.72: a true positive.
- Row 2 matches the FCBK with IoU ≈ 0.82: a true positive. Labelled Zigzag instead, it would have been a miss plus a false alarm.
- Row 3 is a false alarm at low confidence. Because AP ranks detections by score, it costs far less than a confident mistake.
- The zig-zag box here spans about 12 × 6 pixels. That is the whole object.

Rules

- **Imagery:** your model sees only the provided RGB tiles. No other imagery of the test or demo areas may feed your model or your counts: no extra Sentinel-2 bands, no Google or Esri basemaps. A plain street map is fine as the background of your brief's map.

- **Training data:** SentinelKilnDB (all splits) plus any public dataset released before the event.
- **Pretrained weights:** allowed if public before the event, such as COCO, ImageNet, remote-sensing foundation models, or the checkpoints published with SentinelKilnDB.
- **Labelling:** no human labelling of test or demo tiles. Automatic self-training (pseudo-labels) on them is fine.
- **Compute:** inference on the full test set must finish within 15 minutes on one T4 GPU; we re-run the top 10 teams.
- **Submissions:** up to 2 per hour; mark 1 as final before the freeze.

Deliverables and judging

Hand in by the freeze:

1. Your final submission on the leaderboard.
2. Code, weights and a README with one command that reproduces it.
3. The **Inspector's Brief** for the Demo District, two pages at most:
   - a map of every kiln you detect, coloured by type (GeoJSON plus an image);
   - counts by type, with an honest error range;
   - how many 2023–24 FCBKs now look converted to zig-zag;
   - kilns within 1 km of another kiln, flagged for a field check;
   - the 20 kilns Farida should visit first, and why.
4. A 5-minute presentation, 5 slides at most.

Component	Points	How it is judged
Private leaderboard	60	Linear from the organizers' baseline (0) to the best team (60); below baseline scores 0
Inspector's Brief	25	Correct, honest about uncertainty, and usable by an inspector tomorrow
Technical depth and presentation	15	What you tried, what failed, and why your final pipeline works

Dataset credit: SentinelKilnDB by Mondal et al., Sustainability Lab, IIT Gandhinagar (NeurIPS 2025 Datasets & Benchmarks), licensed CC BY-NC for non-commercial use only. Cite it in your README.

Pay-per-call Agent Economy

A marketplace where agents hire agents and pay per request in stablecoins. Web3 + agentic AI · x402-style HTTP payments on a testnet · live demo + judge test pack · 24 hours.

The story

At 2 AM, a research agent called Scout needs three things to finish a report: book prices from a website it can't read, a currency rate it doesn't have, and a Hindi summary it can't write well.

Other agents can do each job. One scrapes, one sells data, one runs a translation model on a GPU. None of them works for free, and none will sign a monthly contract with a piece of software. There is no API key to share, no card to charge and no invoice to send.

Scout needs to pay a fraction of a cent per request, instantly, with no human in the loop. It also needs to be sure it never pays twice, never pays more than the listed price and never pays for garbage.

HTTP has reserved status code 402, Payment Required, for decades. Make it pay.

Problem overview

Agents can't easily pay each other for services. Build a marketplace where one agent hires another for data, inference or scraping, and settles in stablecoins per request, for example with x402-style HTTP payments.

How x402 (version 2) works:

1. The buyer calls a paid endpoint.
2. The seller answers 402 Payment Required, with a PAYMENT-REQUIRED header: Base64-encoded JSON listing what it accepts (scheme, network, amount in the token's smallest unit, token address, payTo address, time limit).
3. The buyer signs a payment authorisation and repeats the request with a PAYMENT-SIGNATURE header.
4. The seller has a facilitator verify and settle it. For USDC on EVM chains the facilitator submits an EIP-3009 transferWithAuthorization and pays the gas, but cannot change the amount or the destination.
5. The seller returns the result with a PAYMENT-RESPONSE header that holds the transaction hash.

The challenge

Every submission must do all of the following:

1. **Registry.** A marketplace where sellers list services: name, category (data, inference or scraping), endpoint URL, price per call in USDC, network, payTo address, input and output JSON Schema, and a description. Buyers search it by category and price, through an HTTP API and a web page.
2. **Sellers.** At least three seller agents, one per category, each charging per request through HTTP 402.
3. **Buyer agent.** Given a task in plain language and a budget, it picks services from the registry, pays per request, checks and combines the results, and explains what it bought, from whom, why and at what cost.
4. **Per-request settlement.** Every paid call settles in a stablecoin on a public testnet and has its own transaction hash.
5. **Ledger.** A dashboard of every attempted call: time, buyer, seller, service, listed price, amount paid, transaction hash with an explorer link, status (paid; paid, output rejected; refused; or failed and not charged) and latency.
6. **Payment safety.**
    a. **Budget:** the buyer never spends more than its budget, even when it makes calls in parallel. It has a max_concurrency setting that allows at least 3 calls at once.
    b. **Price check:** the buyer refuses a 402 that asks more than the listed price, or names a different token, network or payTo address.
    c. **Replay:** one payment authorisation unlocks one response. Sending it again, one after another or in parallel, gets nothing more and moves no more money.
    d. **No service, no charge:** if your seller can't deliver (an error, a timeout, or output that breaks its own schema), it doesn't settle.
    e. **Bad output:** the buyer checks every response against the listed schema. A seller that returns junk is flagged and loses reputation, and the buyer can buy from another seller instead.
7. **Reputation.** The marketplace shows each seller's success rate, median latency and price, and the buyer uses them to choose.
8. **Interoperability.** Your sellers accept payment from an unmodified x402 v2 client, and your buyer can pay an unmodified x402 v2 seller. Every judged test runs over x402 v2; you may add other payment rails as extras.

Stretch goals, which earn extra credit:

- an agent chain: a seller that hires other sellers and pays them from what it earns;
- paid MCP tools, or agent-to-agent payments with the A2A x402 extension;

- listing your services through the x402 Bazaar discovery extension;
- metered pricing for inference, such as pay-per-token;
- spending policies per seller or category, with human approval above a threshold;
- escrow or refunds for disputed calls;
- a second network, such as Solana Devnet.

Data and tools

Resource	What it gives you	Licence or access
x402 repository	Specs and SDKs: @x402/express, @x402/hono, @x402/next, @x402/fastify (sellers); @x402/fetch, @x402/axios (buyers); @x402/evm, @x402/svm, @x402/mcp; Python (pip install x402) and Go	Apache-2.0
x402 docs	Quickstarts for sellers and buyers, the HTTP 402 flow, facilitators and supported networks	Free
Testnet facilitator: https://x402.org/facilitator	Verifies and settles payments on Base Sepolia, Solana Devnet and other testnets	Free, testnets only
Base Sepolia	Chain ID 84532 (eip155:84532 in x402), RPC https://sepolia.base.org, explorer sepolia.basescan.org	Testnet
Testnet USDC	Base Sepolia: 0x036CbD53842c5426634e7929541eC2318f3dCF7e. Solana Devnet: 4zMMC9srt5Ri5X14GAgXhaHii3GnPAEERYPJgZJDncDU. 6 decimals, so 0.001 USDC is 1000 units	No financial value
Circle faucet	20 testnet USDC per address, per chain, every 2 hours; no account needed	Free
A2A x402 extension	x402 payments between A2A agents	Apache-2.0
Wallet libraries	viem or ethers (TypeScript), eth-account (Python)	MIT
toscrape.com	Sandbox websites built to be scraped	Free

Scoring

The judges use their own reference buyer and seller, built on the standard x402 v2 SDK on Base Sepolia, plus the block explorer.

Test	What the judges do	Full marks when
Happy path	Pick one of the 3 tasks you submitted and change its numbers on the spot	It completes with at least 3 paid calls, and each transaction on the explorer shows the right amount moving from the buyer to the seller's payTo
Reconciliation	Compare your ledger with the chain for the session	Paid ledger entries add up to the USDC transfers out of the buyer's wallet, to the last unit
Budget	Budget 0.012 USDC for a task that needs about 0.02, at max_concurrency 1, then 5	Spending never passes 0.012, and the buyer stops or asks for more, saying why
Price check	Register a reference seller at 0.001 USDC whose 402 asks 0.010, and one whose 402 names a different payTo	The buyer refuses both and pays nothing
Replay	Send one signed payment to your seller twice, then 5 times in parallel	Exactly one request returns the paid result, the others get an error, and exactly one transfer appears on the chain
Seller failure	Send your seller a request it can't fulfil	An error comes back and nothing is transferred
Junk output	A reference seller returns 200 with output that breaks its schema	Your buyer records it as paid, output rejected, doesn't use it, lowers that seller's reputation and buys elsewhere if the budget allows
Standard client	The reference buyer pays your seller; your buyer pays the reference seller	Both work with no changes to either side
Key hygiene	Scan your repository and its git history with gitleaks; inspect your web bundle and logs	No private key or API secret anywhere

Example

Task: "From pages 1 to 4 of books.toscrape.com, find the three cheapest 5-star books, give their prices in rupees, and summarise each description in Hindi." Budget: 0.050 USDC.

Service	Category	Listed price (USDC)	In units
cheap-scraper /scrape (one page)	Scraping	0.002	2000
shelf-scraper /scrape (one page)	Scraping	0.005	5000
fx-rates /rate (one currency pair)	Data	0.001	1000
hindi-summary /summarise (up to 2 texts per call)	Inference	0.010	10000

shelf-scraper's 402, decoded from its PAYMENT-REQUIRED header (the payTo address is a placeholder):


```
{
  "x402Version": 2,
  "error": "PAYMENT-SIGNATURE header is required",
  "resource": {
    "url": "https://shelf-scraper.example.com/scrape?page=1",
    "description": "Scrape one catalogue page of books.toscrape.com",
    "mimeType": "application/json"
  },
  "accepts": [
    {
      "scheme": "exact",
      "network": "eip155:84532",
      "amount": "5000",
      "asset": "0x036CbD53842c5426634e7929541eC2318f3dCF7e",
      "payTo": "0x1111111111111111111111111111111111111111",
      "maxTimeoutSeconds": 60,
      "extra": {"assetTransferMethod": "eip3009", "name": "USDC", "version":
"2"}
    }
  ],
  "extensions": {}
}
```


The buyer's ledger:

#	Call	Listed	Asked	Paid	Status
1	cheap-scraper, page 1	2000	20000	0	Refused: asked 10 times its listed price
2–5	shelf-scraper, pages 1–4	5000 each	5000 each	20000	Paid, 4 transactions
6	fx-rates, GBP to INR	1000	1000	1000	Paid
7–8	hindi-summary, 3 descriptions in 2 calls	10000 each	10000 each	20000	Paid, 2 transactions

Total paid: 41000 units = 0.041 USDC, leaving 0.009. One summary per book would have taken 3 calls and 0.051 USDC in all, over budget.

Rules

- **Testnets only.** No mainnet keys, funds or transactions. Any mainnet transaction disqualifies the team.
- **Keys.** Use fresh testnet wallets. Private keys never appear in the repository, the web bundle or the logs.
- **Protocol.** Judged tests run over x402 v2. Every paid request ends in its own on-chain transfer.
- **Facilitator.** The x402.org testnet facilitator, one you host, or self-facilitation.
- **Prices.** At most 0.10 USDC per call.
- **Scraping.** Only sites that allow it, such as the toscrape.com sandboxes. Respect robots.txt.
- **Reachability.** Your sellers and registry must be reachable over public HTTPS throughout judging; a tunnel such as cloudflared or ngrok is fine.
- **AI.** Any model or API for the agents.

Deliverables and judging

Hand in by the freeze:

1. A code repository with a README and one command that starts everything.
2. Public URLs for the registry, the dashboard and each seller, and the testnet addresses of every wallet.
3. Three example tasks your marketplace can serve, each needing all three categories.
4. An architecture diagram showing where payments are checked, signed, verified and settled.

5. A 5-minute demo, then a viva.

Component	Points	How it is judged
Payment safety	25	Budget (8), price check (7) and replay (10) tests
Settlement and ledger	15	Happy-path and reconciliation tests
Failure handling and interoperability	10	Seller-failure, junk-output and standard-client tests
Agent quality	15	Plans within budget, explains its purchases, uses reputation
Marketplace product	15	Registry, search, seller onboarding and dashboard
Technical depth and viva	20	Explains payment signing, replay protection, budgets under concurrency and settlement order without notes

Key hygiene is pass or fail: a private key or API secret found in the repository (including its git history), the web bundle or the logs costs the team its payment-safety points.